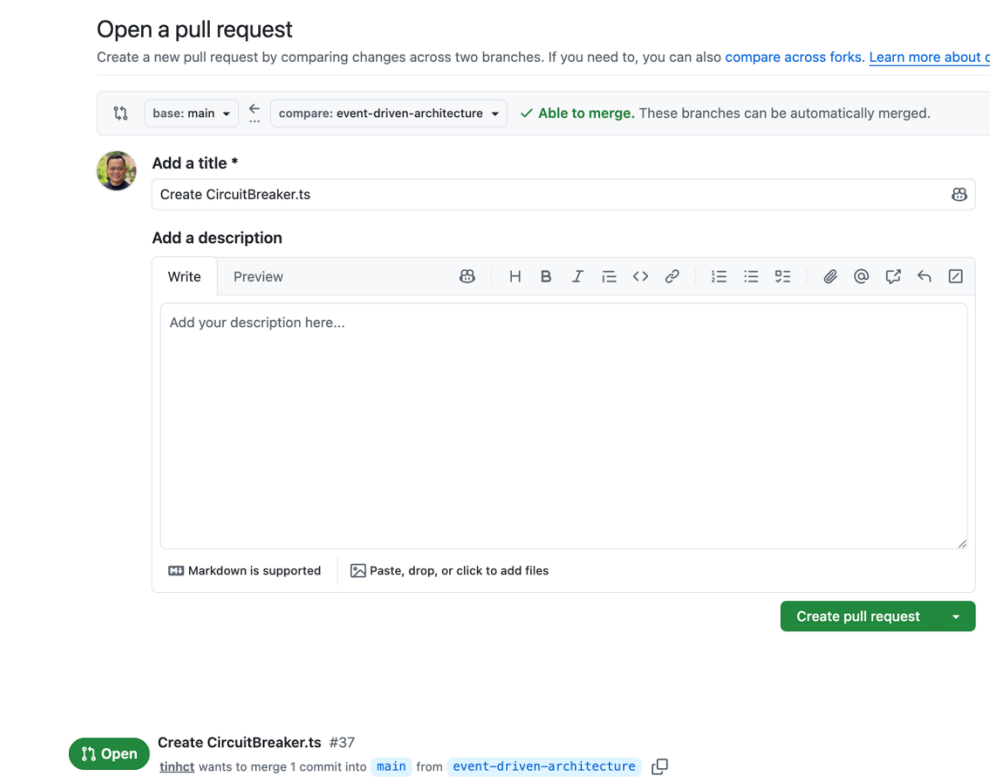
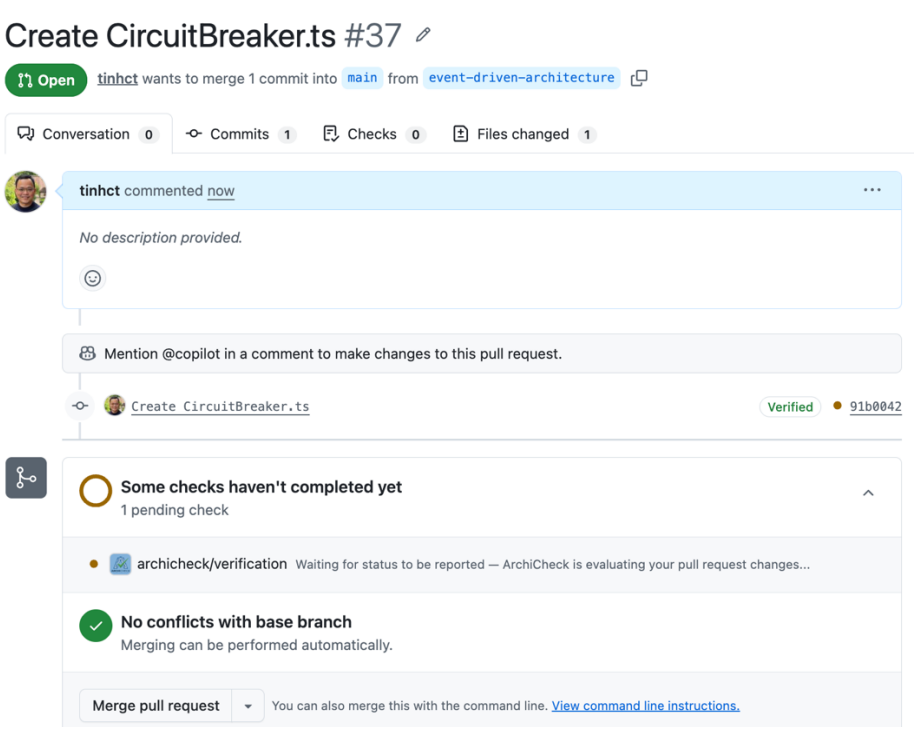
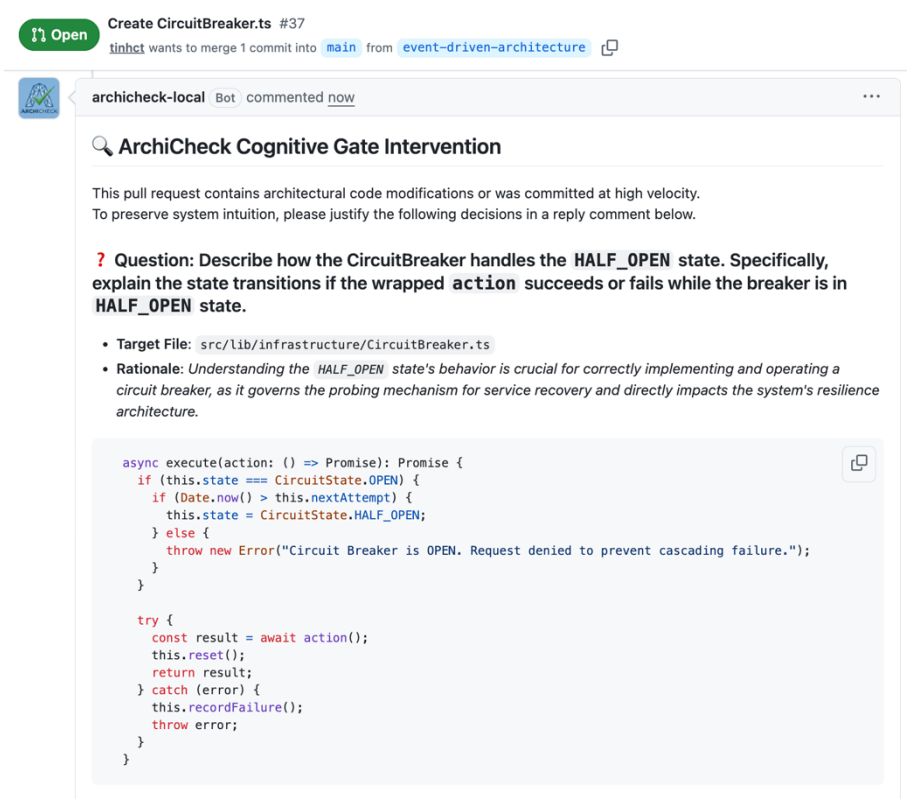
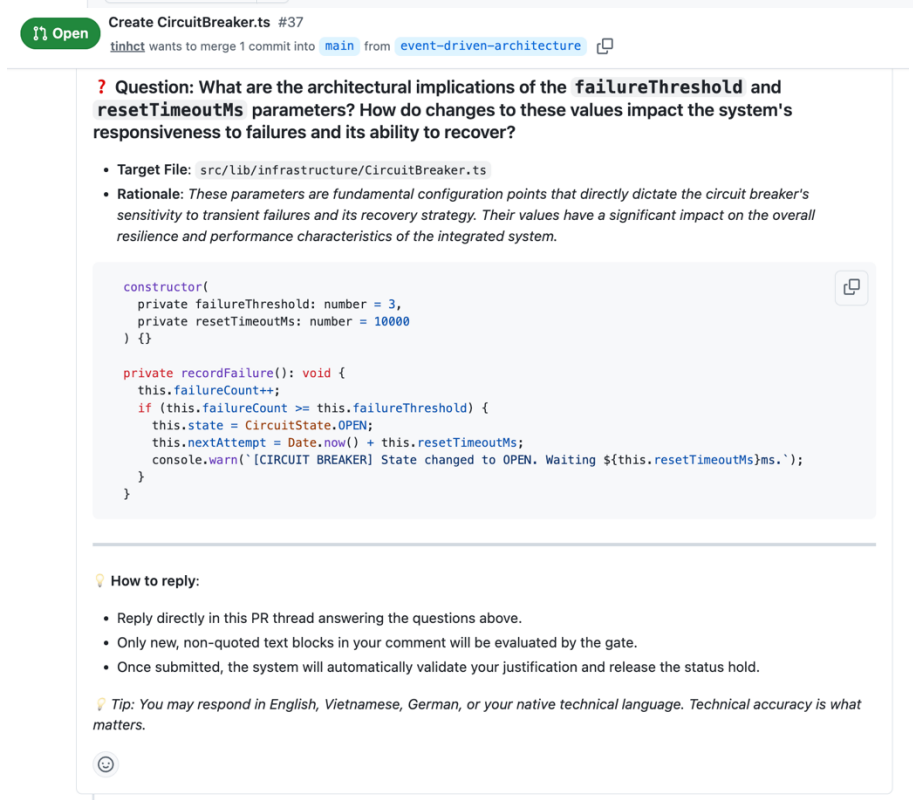
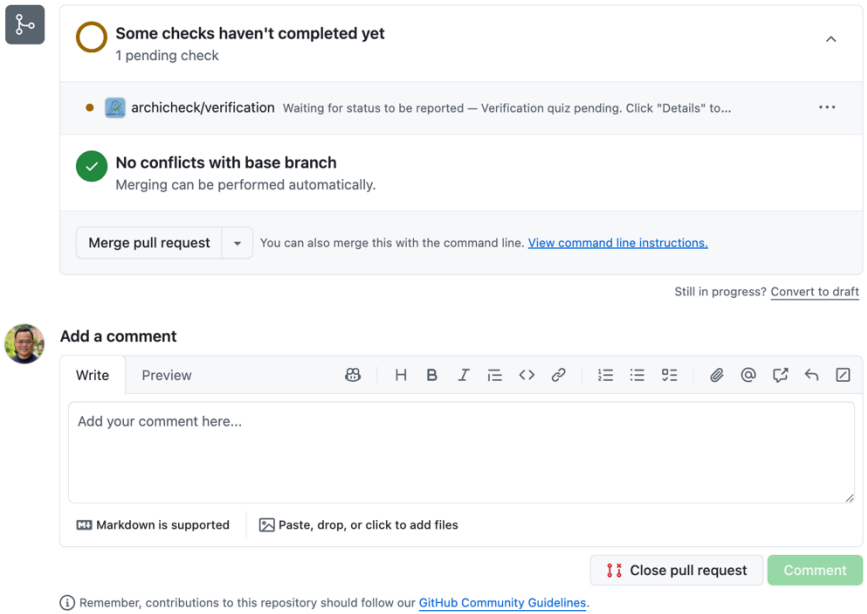
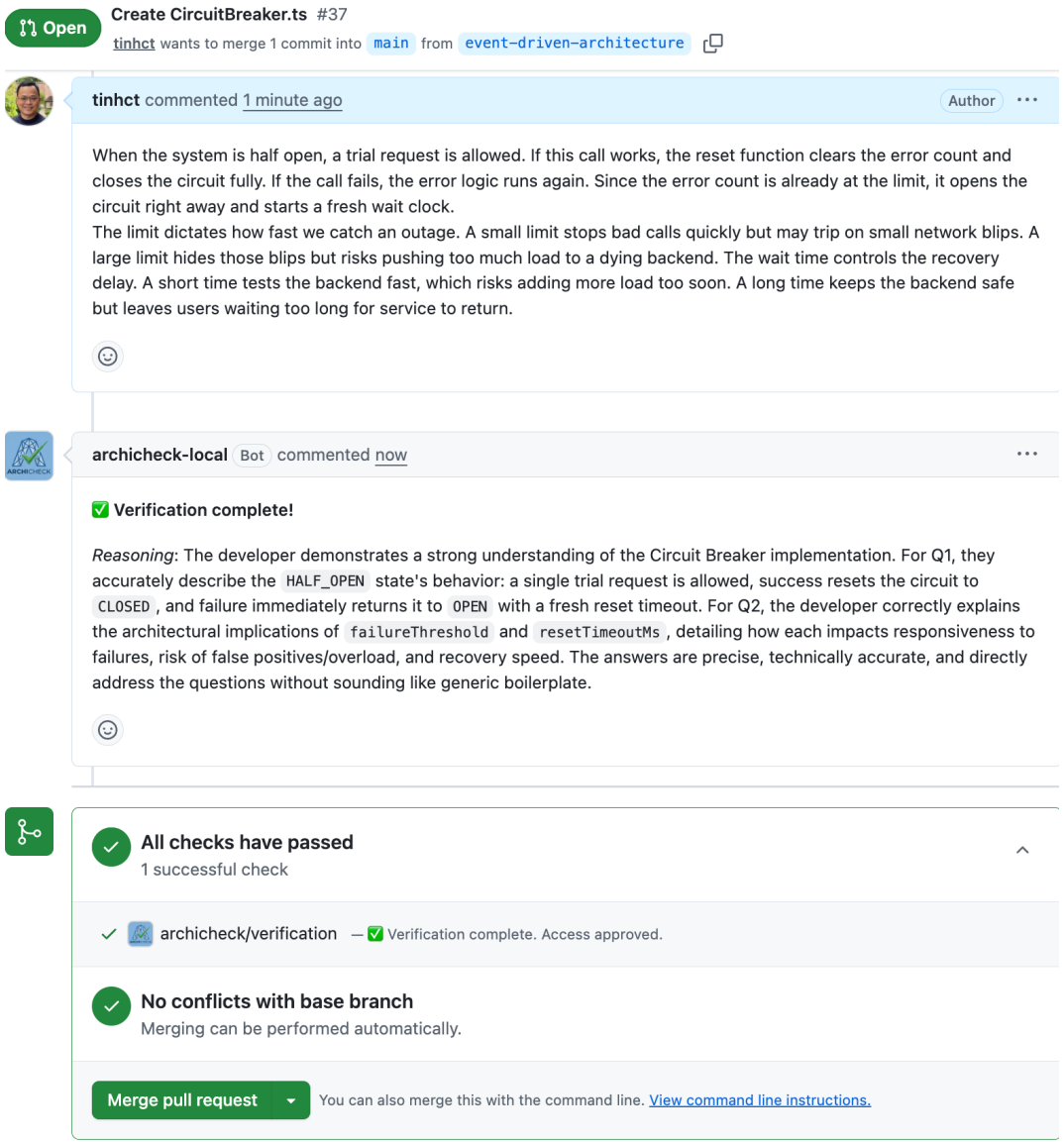

Open a pull request
Create a new pull request by comparing changes across two branches. If you need to, you can also compare across forks. Learn more about c
base: main
compare: event-driven-architecture
Able to merge. These branches can be automatically merged.
Add a title *
Create CircuitBreaker.ts
Add a description
Write
Preview
Add your description here...
Markdown is supported
Paste, drop, or click to add files
Create pull request


Create CircuitBreaker.ts #37
Open
tinhct wants to merge 1 commit into main from event-driven-architecture
Conversation 0
Commits 1
Checks 0
Files changed 1
tinhct commented now
No description provided.
Mention @copilot in a comment to make changes to this pull request.
Create CircuitBreaker.ts
Verified
91b0042
Some checks haven't completed yet
1 pending check
archicheck/verification Waiting for status to be reported — ArchiCheck is evaluating your pull request changes...
No conflicts with base branch
Merging can be performed automatically.
Merge pull request
You can also merge this with the command line. View command line instructions.


Open
Create CircuitBreaker.ts #37
tinhct wants to merge 1 commit into main from event-driven-architecture
archicheck-local Bot commented now
ArchiCheck Cognitive Gate Intervention
This pull request contains architectural code modifications or was committed at high velocity.
To preserve system intuition, please justify the following decisions in a reply comment below.
❓ Question: Describe how the CircuitBreaker handles the HALF_OPEN state. Specifically, explain the state transitions if the wrapped action succeeds or fails while the breaker is in HALF_OPEN state.
Target File: src/lib/infrastructure/CircuitBreaker.ts
Rationale: Understanding the HALF_OPEN state's behavior is crucial for correctly implementing and operating a circuit breaker, as it governs the probing mechanism for service recovery and directly impacts the system's resilience architecture.
async execute(action: () => Promise): Promise {
if (this.state === CircuitState.OPEN) {
if (Date.now() > this.nextAttempt) {
this.state = CircuitState.HALF_OPEN;
} else {
throw new Error("Circuit Breaker is OPEN. Request denied to prevent cascading failure.");
}
}
try {
const result = await action();
this.reset();
return result;
} catch (error) {
this.recordFailure();
throw error;
}
}


Open
Create CircuitBreaker.ts #37
tinhct wants to merge 1 commit into main from event-driven-architecture
❓ Question: What are the architectural implications of the failureThreshold and resetTimeoutMs parameters? How do changes to these values impact the system's responsiveness to failures and its ability to recover?
Target File: src/lib/infrastructure/CircuitBreaker.ts
Rationale: These parameters are fundamental configuration points that directly dictate the circuit breaker's sensitivity to transient failures and its recovery strategy. Their values have a significant impact on the overall resilience and performance characteristics of the integrated system.
constructor(
private failureThreshold: number = 3,
private resetTimeoutMs: number = 10000
) {}
private recordFailure(): void {
this.failureCount++;
if (this.failureCount >= this.failureThreshold) {
this.state = CircuitState.OPEN;
this.nextAttempt = Date.now() + this.resetTimeoutMs;
console.warn(`[CIRCUIT BREAKER] State changed to OPEN. Waiting ${this.resetTimeoutMs}ms.`);
}
}
How to reply:
Reply directly in this PR thread answering the questions above.
Only new, non-quoted text blocks in your comment will be evaluated by the gate.
Once submitted, the system will automatically validate your justification and release the status hold.
Tip: You may respond in English, Vietnamese, German, or your native technical language. Technical accuracy is what matters.


Some checks haven't completed yet
1 pending check
archicheck/verification Waiting for status to be reported — Verification quiz pending. Click "Details" to...
No conflicts with base branch
Merging can be performed automatically.
Merge pull request
You can also merge this with the command line. View command line instructions.
Still in progress? Convert to draft
Add a comment
Write
Preview
Add your comment here...
Markdown is supported
Paste, drop, or click to add files
Close pull request
Comment
Remember, contributions to this repository should follow our GitHub Community Guidelines.


Open
Create CircuitBreaker.ts #37
tinhct wants to merge 1 commit into main from event-driven-architecture
tinhct commented 1 minute ago
Author
When the system is half open, a trial request is allowed. If this call works, the reset function clears the error count and closes the circuit fully. If the call fails, the error logic runs again. Since the error count is already at the limit, it opens the circuit right away and starts a fresh wait clock.
The limit dictates how fast we catch an outage. A small limit stops bad calls quickly but may trip on small network blips. A large limit hides those blips but risks pushing too much load to a dying backend. The wait time controls the recovery delay. A short time tests the backend fast, which risks adding more load too soon. A long time keeps the backend safe but leaves users waiting too long for service to return.
archicheck-local Bot commented now
✅ Verification complete!
Reasoning: The developer demonstrates a strong understanding of the Circuit Breaker implementation. For Q1, they accurately describe the HALF_OPEN state's behavior: a single trial request is allowed, success resets the circuit to CLOSED, and failure immediately returns it to OPEN with a fresh reset timeout. For Q2, the developer correctly explains the architectural implications of failureThreshold and resetTimeoutMs, detailing how each impacts responsiveness to failures, risk of false positives/overload, and recovery speed. The answers are precise, technically accurate, and directly address the questions without sounding like generic boilerplate.
All checks have passed
1 successful check
archicheck/verification — ✅ Verification complete. Access approved.
No conflicts with base branch
Merging can be performed automatically.
Merge pull request
You can also merge this with the command line. View command line instructions.